

Assignment 3: ASR

EE6130: Advanced Topics in Signal Processing

Nishchala Mukku (EE24S004)

November 12, 2024

1 Question

Tasks for Speaker Diarization:

- Finetune the openai/whisper-tiny model with the Assamese data from the mozilla-foundation/common_voice_11_0 dataset.

Write a two page report on your findings

2 Solution - My Work

Kaggle Notebook link: [Link](#).

3 Fine-Tuning Whisper Model for ASR: Steps and Methodology

1. Dataset Preparation

- **Dataset Selection:** Selected a suitable dataset (e.g., Common Voice) with audio and transcription data in the target language.
- **Dataset Loading:** Used Hugging Face's `datasets` library to load and inspect the dataset.
- **Data Cleaning and Preprocessing:** Removed unnecessary columns and formatted the dataset to include only essential fields like audio and text. Resampled all audio files to 16kHz, as required by Whisper.

2. Feature Extraction and Tokenization

- **WhisperProcessor:** Initialized `WhisperProcessor` to handle both audio feature extraction and text tokenization in a single step.
- **Audio Feature Extraction:** Used `WhisperFeatureExtractor` to convert audio into log-Mel spectrogram features.

- **Text Tokenization:** Tokenized transcriptions using `WhisperTokenizer` with parameters such as language and task type (e.g., `task="transcribe"`).

3. Data Preparation for Model Training

- **Data Collator:** Implemented a custom data collator to pad and batch the input features and target labels separately.
- **Gradient Accumulation:** Used gradient accumulation to simulate a larger batch size on limited GPU memory.

4. Model Initialization and Configuration

- **Loading the Model:** Initialized `WhisperForConditionalGeneration` from the pre-trained "openai/whisper-tiny" checkpoint.
- **Model Configuration:** Customized settings such as `forced_decoder_ids=None` and `suppress_tokens=[]`.
- **Device Allocation:** Assigned the model to GPU if available.

5. Training Setup and Hyperparameter Tuning

- **Seq2SeqTrainingArguments:** Defined key hyperparameters, including:
 - `learning_rate`: Experimented with values (e.g., `1e-5` and `5e-6`) to find optimal learning rates.
 - `num_train_epochs`: Tested different epoch counts.
 - `gradient_accumulation_steps`: Adjusted for simulating larger batch sizes.
 - `fp16`: Enabled mixed-precision training.
 - `evaluation_strategy`: Set to "steps" for periodic evaluation.
- **Trainer Initialization:** Configured `Seq2SeqTrainer` with the model, data collator, training arguments, and WER calculation.

6. Evaluation Metric (WER)

- **Word Error Rate (WER):** Defined `compute_metrics` function to calculate WER after each evaluation step.
- **Metric for Best Model:** Set WER as the `metric_for_best_model` to track the model with the lowest WER.

7. Training and Evaluation

- **Model Training:** Trained the model with `trainer.train()` while monitoring WER.
- **Validation:** Performed evaluations at intervals (e.g., every 50 steps) to assess generalization.

8. Saving and Exporting the Model

- **Model Checkpoints:** Saved periodic checkpoints (e.g., every 500 steps).
- **Final Model Save:** Saved the fine-tuned model with `trainer.save_model()` for deployment or evaluation.
- **Model Compression:** Used `zip` to archive model files for sharing or download.

9. Results Analysis and Insights

- **Hyperparameter Impact:** Analyzed variations in learning rate, gradient accumulation, and epochs on WER.
- **Model Performance:** Assessed WER to determine if accuracy met desired benchmarks.
- **Future Improvements:** Noted potential for further fine-tuning, such as more epochs, additional hyperparameter tuning, or using larger Whisper models.

Model	Learning Rate	num train epochs	Gradient Accumulation steps	Insights
1	1×10^{-5}	10	1	Baseline setup to gauge basic performance.
2	1×10^{-5}	20	1	Increased epochs to check if more training time improves WER or causes overfitting.
3	1×10^{-5}	10	4	Gradient accumulation (4 steps) simulates a larger batch size for better stability and efficient memory usage.
4	1×10^{-5}	5	4	Short experiment for pipeline validation; limited steps may not yield meaningful results.
5	5×10^{-6}	10	1	Lower learning rate with fewer epochs, intended to reduce overfitting; may need more epochs to converge.
6	1×10^{-5}	20	2	Minimal steps likely for quick setup testing, rather than training.
7	5×10^{-6}	40	4	Lower learning rate with increased epochs and gradient accumulation for gradual, refined learning.

Table 1: Summary of Model Configurations and Insights for Whisper Fine-tuning